МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

КУРСОВАЯ РАБОТА

по дисциплине: Настройка и администрирование сервисного программного обеспечения

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: Настройка и администрирование платформы для автоматизации учёта рабочих часов и задач сотрудников

Студент: Данов Арсений Иванович

Группа: ИКБО-10-23

Работа представлена к защите_____(дата)_____/Данов А.И./

(подпись и ф.и.о. студента)

Руководитель: Ст. преп. Русляков Алексей Александрович

Работа допущена к защите_____(дата)_____/Русляков А.А./

(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: _____

_____ / _____ /

_____ / _____ /

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, принявших защиту)

М. РТУ МИРЭА. 2024

Заведующему кафедрой

инструментального и прикладного программного обеспечения

(ИиППО)

Института информационных технологий (ИТ)

Болбакову Роману Геннадьевичу

От студента Данова Арсения Ивановича

_____

ФИО

ИКБО-10-23

группа

2 курс

курс

Контакт: *Danov2005@yandex.ru*

Заявление

Прошу утвердить мне тему *курсовой работы* по дисциплине «*Настройка и администрирование сервисного программного обеспечения*» образовательной программы бакалавриата *09.03.04 (Программная инженерия)*

Тема: Настройка и администрирование платформы для автоматизации учёта рабочих часов и задач сотрудников.

Приложение: лист задания на КР в 2-ух экземплярах на двухстороннем листе (проект)

Подпись студента _____ / _____

подпись *ФИО*

Дата _____

Подпись руководителя _____ / _____

подпись *Должность, ФИО*

Дата _____

Согласовано Зав. кафедрой ИиППО _____ / Болбаков Р.Г.

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Настройка и администрирование сервисного программного обеспечения
по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Данов Арсений Иванович
Группа: ИКБО-10-23
Срок представления к защите: 02.12.2024
Руководитель: Русляков Алексей Александрович, ст. преподаватель

Тема: Настройка и администрирование платформы для автоматизации учёта рабочих часов и задач сотрудников.

Исходные данные: используемые технологии: docker, docker compose, docker swarm, kubernetes, dockerfile, docker-compose.yaml, редактор кода Visual Studio Code, наличие: сетевого взаимодействия контейнеров: технологий CI/CD, кластера. Нормативный документ: инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Провести анализ предметной области контейнерной платформы на базе docker. 2.Обосновать выбор технологий разработки контейнерной платформы на базе docker. 3. Создать пять и более контейнеров. 4. Создать кластерную архитектуру. 5. Реализовать CI/CD . 6. Организовать деплой созданной инфраструктуры. 7. Настроить сетевое межконтейнерное взаимодействие. 8. Создать презентацию по выполненной курсовой работе..

Руководителем проведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: _____/Р. Г. Болбаков/, «16» 09 2024 г.

Задание на КР выдал: _____ / А.А. Русляков /, «16» 09 2024 г.

Задание на КР получил: _____ / А.И. Данов /, «16» 09 2024 г.

РЕФЕРАТ

Отчёт 40 с., 17 рис., 14 ист.

АДМИНИСТРИРОВАНИЕ, РАЗВЕРТЫВАНИЕ, КОНТЕЙНЕРИЗАЦИЯ, БАЛАНСИРОВКА, ПРОКСИРОВАНИЕ

Данов А.И., Курсовая работа направления подготовки «Программная инженерия» предмета «Настройка и администрирование сервисного программного обеспечения» на тему «Настройка и администрирование платформы для автоматизации учёта рабочих часов и задач сотрудников»: М. 2024 г., МИРЭА – Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра инструментального и прикладного программного обеспечения (ИиППО)

Целью работы является настройка и администрирование платформы для автоматизации учёта рабочих часов и задач сотрудников. Проанализирована предметная область, разработана архитектура приложения, выбраны технологии реализации компонентов, реализованы настройка и развертывание приложения, описаны теоретические возможности расширения настройки.

СОДЕРЖАНИЕ

Определения, обозначения и сокращения 6
Введение 8
1 Анализ предметной области 11
1.1 Анализ теоретической базы 11
1.2 Анализ конкурентных решений 12
1.2.1 Достоинства и недостатки существующих решений 13
1.3 Обоснование выбора технологий 14
1.3.1 Формирование требований к приложению 14
1.3.2 Обоснование выбора Docker 15
2 Архитектура приложения 17
2.1 Структурные элементы приложения 17
2.1.1 Взаимосвязь компонентов приложения 18
2.1.2 Выбор технологий реализации контейнеров 19
3 Реализация программного продукта 23
3.1 Практическая реализация продукта 23
3.1.1 Реализация архитектуры контейнеров 23
3.1.2 Реализация наполнения контейнеров 24
3.2 Теоретическая реализация продукта 32
3.3 Инструкция к развертыванию приложения 34
3.4 Демонстрация работоспособности приложения 36
Заключение 38
Список использованных источников 39

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчёте применяют следующие определения, обозначения и сокращения.

API (Application Programming Interface)	– программный интерфейс, определяющий способы взаимодействия компонентов программного обеспечения.
Argon2	– алгоритм хеширования паролей, устойчивый к атакам с использованием GPU и ASIC.
Clean Architecture	– архитектурный паттерн, предложенный Робертом Мартином, обеспечивающий независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов.
DDD (Domain-Driven Design)	– подход к проектированию программного обеспечения, ориентированный на моделирование предметной области и использование единого языка (Ubiquitous Language).
Dependency Injection (DI)	– паттерн проектирования, при котором зависимости компонентов передаются извне, а не создаются внутри компонента.
DTO (Data Transfer Object)	– объект передачи данных, используемый для обмена структурированной информацией между слоями приложения.
ER-диаграмма (Entity-Relationship Diagram)	– диаграмма «сущность–связь», используемая для моделирования структуры данных и отношений между сущностями.
FastAPI	– современный веб-фреймворк для Python, ориентированный на создание REST API с поддержкой асинхронности, автоматической валидации данных и документирования.
HTTP (HyperText Transfer Protocol)	– протокол передачи данных между клиентом и сервером в веб-среде.

JWT (JSON Web Token)	– компактный и самодостаточный формат для передачи данных в виде JSON-объекта, используемый для аутентификации.
LMS (Learning Management System)	– система управления обучением, предназначенная для организации образовательного процесса, хранения материалов и контроля результатов.
MVC (Model-View-Controller)	– архитектурный паттерн, разделяющий приложение на три взаимосвязанных компонента: модель, представление и контроллер.
ORM (Object-Relational Mapping)	– технология отображения объектных структур в реляционные базы данных.
Repository	– паттерн, инкапсулирующий логику доступа к данным и предоставляющий интерфейс для операций с сущностями.
REST (Representational State Transfer)	– архитектурный стиль взаимодействия распределённых систем, использующий принципы унифицированного интерфейса и обмена представлениями ресурсов.
Service Layer	– слой бизнес-логики приложения, обрабатывающий прикладные операции и обращающийся к репозиториям.
SQL (Structured Query Language)	– язык запросов, используемый для управления реляционными базами данных.
SQLAlchemy	– библиотека ORM и инструмент работы с реляционными базами данных в Python.
СУБД	– система управления базами данных.
Фреймворк	– программный каркас, упрощающий разработку приложений и задающий архитектурные принципы.

ВВЕДЕНИЕ

На сегодняшний день многие компании переходят от традиционной системы фиксированной оплаты труда к более гибким и справедливым подходам. Одним из таких решений является оплата на основе реального количества отработанных часов. Такой подход позволяет учитывать индивидуальный вклад каждого сотрудника и мотивировать его на достижение лучших результатов.

Однако внедрение этой системы требует использования эффективных инструментов для учета рабочего времени. Ручной подсчет не только увеличивает риск ошибок, но и отнимает значительное количество времени у бухгалтерии и менеджеров. В условиях повсеместной автоматизации, охватывающей практически все процессы в современных организациях, такие методы становятся все менее актуальными. Чтобы избежать трудностей, связанных с ручным учетом, компании всё чаще внедряют автоматизированные платформы для учета рабочего времени.

Автоматизация этого процесса позволяет исключить человеческий фактор при сборе данных, предоставляя руководству актуальную информацию о производительности сотрудников. Благодаря этому руководители могут оперативно реагировать на возникающие проблемы, принимать взвешенные решения и эффективно распределять ресурсы. Кроме того, системы автоматизированного учета облегчают интеграцию данных с другими корпоративными процессами, такими как начисление заработной платы или планирование рабочего графика.

Такие платформы также играют важную роль в обеспечении прозрачности и справедливости внутри коллектива. Кроме того, внедрение подобных решений помогает организациям соответствовать требованиям трудового законодательства, связанным с нормированием и контролем труда.

В рамках данной курсовой работы была разработана и частично реализована платформа для автоматизации учёта рабочих часов и задач сотрудников. Основное внимание уделялось построению архитектуры, способной обеспечивать надёжность, масштабируемость и гибкость системы, а также упрощать её развёртывание и поддержку.

Был разработан бэкенд-сервис, выполняющий ключевые функции платформы: управление пользователями и их учёт рабочего времени и задач.

Для взаимодействия с пользователями создан фронтенд, предоставляющий возможность авторизации в качестве сотрудника или руководителя. Сотрудникам доступно внесение данных о выполненных задачах и затраченном на их выполнение времени. Руководителям предоставлен интерфейс для просмотра статистики сотрудников, информация о выполненных ими задачах, возможность ознакомиться с комментариями, отчётами и общей суммой выплат по итогам работы сотрудников.

Вся архитектура системы реализована на базе контейнерной платформы Docker. Это позволило значительно упростить процесс развёртывания приложений, обеспечив изоляцию компонентов и их независимость друг от друга. Для маршрутизации и балансировки нагрузки использовался Traefik, что обеспечило равномерное распределение запросов между сервисами и повысило отказоустойчивость системы.

Также были рассмотрены теоретические возможности дальнейшего расширения спроектированной архитектуры:

– Внедрение CI/CD для автоматизации процессов тестирования, сборки и развёртывания для ускорения доставки обновлений и улучшения стабильности;

– Кластеризация с использованием Kubernetes для обеспечения масштабируемости и высокой доступности системы при росте нагрузки;

– Реализация API путём создания интерфейсов для интеграции с внешними сервисами и корпоративными системами.

Цель работы — разработка, настройка и администрирование платформы для автоматизации учета рабочих часов и задач сотрудников.

Для достижения поставленной цели были поставлены следующие задачи:

– Анализ предметной области;

– Формирование требований к платформе по автоматизации рабочих часов и задач сотрудников;

– Формирование концепций проектируемой платформы и архитектуры приложения;

– Выбор инструментов разработки и настройки платформы и её составных компонентов;

– Разработка и настройка платформы на базе технологий контейнеризации;

– Выработка рекомендаций по процессу настройки и развертывания платформы.

Объект исследования — системы учёта рабочих часов и задач сотрудников.

Предмет исследования — автоматизация учёта рабочих часов и задач сотрудников на базе платформы на основе технологий контейнеризации.

1 Анализ предметной области

1.1 Анализ теоретической базы

Контейнерная платформа Docker предоставляет инструменты развертывания и управления приложениями на базе контейнеров. Контейнеризация позволяет упаковывать приложение вместе с его завимимостями и виртуальным окружением в легковесный контейнер, который возможно быстро и просто развернуть на любой инфраструктуре. Легковесность контейнера достигается засчёт использования ядра операционной системы, на которой разворачивается приложение [1].

Ручные создание и настройка контейнеров являются нетривиальными задачами, в ходе исполнения которых возможны частые ошибки. Для минимизации совершаемых ошибок и оптимизации процесса разработки приложений, ускорения и упрощения их развертывания существуют контейнерные платформы, берущие на себя низкоуровневые задачи по созданию контейнеров и их связи с ядром операционной системы.

Контейнерная платформа Docker является одним из наиболее популярных решений для развертывания и управления приложениями в изолированных средах. Образы, на основе которых создаются контейнеры, являются практически полностью готовым компонентом системы, которой перед применением необходимо настроить для достижения желаемых результатов.

Для множества задач уже существуют готовые официальные образы, позволяющие сразу использовать необходимую технологию без самостоятельной разработки нового решения или переработки и контейнеризации существующего. Примером таких образов могут являться образы баз данных PostgreSQL, MySQL, Redis или веб-серверы Nginx и Apache.

Docker предоставляет мощные инструменты для работы с контейнерами, включая создание и управление образами, настройку сетей. Ещё одним немаловажным фактором является интеграцию с репозиториями контейнеров, такими как Docker Hub.

Репозитории контейнеров позволяют реализовывать систему контроля версий, хранить и получать доступ ко множество готовых официальных

и неофициальных образов, загружать и скачивать собственные образы, созданные с нуля или на базе существующих.

1.2 Анализ конкурентных решений

Docker является лидером в области контейнеризации. По данным опроса от StackOverflow более 50% профессиональных разработчиков используют в своей работе именно Docker [2].

Однако существуют и другие популярные решения, предлагающие схожий функционал. Среди них можно выделить Podman, LXC/LXD и CRI-O, которые активно используются в зависимости от задач и требований инфраструктуры. Эти платформы имеют свои особенности, принципы работы и подходы к организации контейнеров.

Podman позиционируется как альтернатива Docker, построенная на схожих принципах работы с контейнерами и образами. Главным отличием Podman является отсутствие центрального демона (Docker Daemon), что делает его архитектуру более распределённой и безопасной. Контейнеры в Podman запускаются как независимые процессы, что упрощает управление и повышает отказоустойчивость системы. Поддержка OCI-образов позволяет использовать Podman совместно с Docker и другими совместимыми решениями [3].

LXC/LXD представляет собой более традиционный подход к контейнеризации, ориентированный на изоляцию операционных систем. В отличие от Docker, который сосредоточен на приложениях, LXC/LXD предоставляет окружение, ближе к полноценным виртуальным машинам. Это делает их удобными для работы с более сложными и монолитными приложениями, которые требуют полной операционной среды. LXD, как расширение LXC, добавляет удобные интерфейсы для управления контейнерами и упрощает их развертывание [4].

CRI-O разработан как специализированное решение для интеграции с Kubernetes и соответствует требованиям CRI. В отличие от Docker, CRI-O оптимизирован для работы с контейнерными оркестраторами, предоставляя минимальный набор функций, необходимых для запуска контейнеров. Он использует OCI-образы и контейнеры, обеспечивая совместимость с другими решениями, такими как Docker и Podman [5].

Хотя представленные конкурентные решения и могут являться заменой Docker, довольно значительные отличия всё же не позволяют называть их полностью взаимозаменяемыми. У каждой платформы есть более приоритетные и оптимальные задачи, которые она решает, уступая конкурентам в других ситуациях.

1.2.1 Достоинства и недостатки существующих решений

Docker является одной из самых популярных платформ для контейнеризации. Популярность он приобрёл вследстве своей простоты и удобства, а благодаря популярности вокруг Docker сформировалось активное сообщество и появилось множество доступных ресурсов помогающих в его изучении. Как результат, вышеописанные факторы сделали Docker стандартом в области контейнеризации [6].

Достоинства:

– Простота освоения и использования, понятные интерфейсы и комманды;

– Поддержка стандартов OCI и совместимость с другими технологиями;

– Широкий набор инструментов (Docker Compose, Docker Swarm);

– Популярность среди разработчиков ПО и большое количество готовых образов.

Недостатки:

– Зависимость от центральной службы Docker Daemon, приводящая к возможным уязвимостям;

– Ограничения производительности и гибкости;

– Необходимость дополнительных технологий для сложных сценариев оркестрации.

Podman предоставляет схожий с Docker функционал, но в его реализации отсутствует центральный демон, что делает архитектуру более безопасной.

Достоинства:

– Отсутствие центрального демона повышает безопасность и отказоустойчивость;

– Полная совместимость с Docker-образами и инструментами;

– Возможность работы без привилегий суперпользователя.

Недостатки:

– Менее развитая экосистема и сообщество по сравнению с Docker;

– Необходимость дополнительных настроек для некоторых функциональных возможностей, уже встроенных в Docker.

LXC/LXD предоставляет традиционный подход к контейнеризации, фокусируясь на изоляции на уровне операционных систем. Реализация LXC/LXD по своей структуре ближе к виртуализации и эмуляции операционных систем.

Достоинства:

– Полная изоляция, позволяющая запускать контейнеры как полноценные виртуальные машины;

– Удобство в работе с монолитными или устаревшими приложениями, требующими полноценной ОС.

Недостатки:

– Менее подходящий выбор для микросервисных архитектур в силу большей громоздкости контейнеров;

– Ограниченная поддержка стандартов OCI.

CRI-O является лёгкой средой выполнения контейнеров, специально созданной для Kubernetes и имеющей с ним более высокую интеграцию. А также и соответствует стандарту CRI.

Достоинства:

– Оптимизация для работы с Kubernetes;

– Минимальная нагрузка на систему благодаря лёгкой архитектуре;

– Поддержка OCI-образов для совместимости с другими платформами.

Недостатки:

– Ограниченный функционал, фокусирующийся только на Kubernetes;

– Меньше возможностей для работы вне экосистемы Kubernetes.

1.3 Обоснование выбора технологий

1.3.1 Формирование требований к приложению

Разрабатываемая платформа для автоматизации учёта рабочих часов и задач сотрудников должна на базовом уровне удовлетворять следующим условиям:

– Предоставлять независимый доступ для сотрудников и руководителей, быть легкодоступной;

– Обладать высокой стабильностью и отказоустойчивостью для беспрерывной работы сервиса;

– Равномерно распределять нагрузку между внутренними компонентами для минимизирования задержек в работе пользователя;

– Иметь возможность простого и быстрого развертывания на любой машине хосте.

Исходя из перечисленных выше требований можно сделать вывод, что разрабатываемое приложение должно быть реализовано с применением технологий контейнирезации для обеспечения простого и быстрого развертывания. Приложение должно иметь возможность поднимать реплики существующих контейнеров для равномерного распределения нагрузки между копиями. Платформа должна быть развернута в облаке или на выделенной машине/хостинге. Для обеспечения отказоустойчивости приложение должно обладать механизмом проксирования запросов извне.

1.3.2 Обоснование выбора Docker

Исходя из всего вышесказанного в качестве контейнерной платформы для разработки данного приложения был выбран Docker, как обладающий наибольшим набором существенных преимуществ и наименее существенными недостатками.

Docker является стандартом в области контейнеризации и обладает высокой совместимостью с современными технологиями. Docker предоставляет удобные инструменты для упаковки, развертывания и управления приложениями, что позволит в ходе разработки приложения сосредоточиться на создании функциональности.

Также благодаря всецелой поддержке стандартов OCI будет обеспечена совместимость с другими инструментами и возможность переноса приложения между различными платформами.

Одним из ключевых преимуществ Docker является простота работы с контейнерами. Благодаря интуитивно понятным командам и большому количеству готовых образов в Docker Hub, процесс развертывания приложения

становится быстрым и эффективным. Это особенно важно для проектов, где требуется частое обновление и тестирование, так как контейнеры могут быть перезапущены или заменены без необходимости перезагрузки всей системы.

2 Архитектура приложения

2.1 Структурные элементы приложения

Исходя из сформированных в главе 1 требований к платформе автоматизации учёта рабочих часов и задач сотрудников необходимо определить структурные элементы, из которых будет состоять разрабатываемое приложение.

Так как для платформы крайне важный удобный и быстрый доступ для сотрудников, было решено реализовать её в качестве веб приложения, доступ к которому возможен благодаря сети интернет из любой точки. Для этого необходимо будет реализовать деплой созданного приложения на выделенный персональный сервер или хостинг, благодаря возможности гибкой настройки доступа которых также будет возможно ограничить доступ к приложению для внешних пользователей.

Но даже без ограничения доступа на уровне хостинга крайне важно реализовать раздельный доступ для сотрудников и руководителей, а также отдельный доступ только к необходимым данным каждого сотрудника и руководителя в частности. Из этого следует вывод о необходимости реализации системы авторизации и аутентификации в рамках приложения. Для этого, а также для хранения другой информации о сотрудниках, о выполняемых ими задачах и для учёта рабочих часов приложение должно обладать компонентом базы данных, который, ввиду разработки с применением технологий контейниризации, должен быть, как и все компоненты, реализован отдельным независимым (в плоскости виртуального окружения и зависимостей) контейнером.

Для обработки действий пользователей, выполнения логических и функциональных действий, внесения изменений в базу данных и чтения данных из неё, а также для обслуживния приходящих от пользователей запросов приложение должно обладать компонентом бекенда.

Для корректной работы практически любого бекенда необходим веб-сервер, берущий часть запросов на себя. В самом простом варианте необходимо как минимум реализовать веб-сервером отдачу статических и медиа файлов, чтобы не нагружать бессмысленными одинаковыми запросами бекенд.

Считается хорошей практикой разделение прямых запросов пользователя и бекенда/веб-сервера проксирующим элементом. Это не только позволит при необходимости внедрить дополнительную логику маршрутизации, но и позволит развязать источники множественных запросов (потенциальный наплыв пользователей или же DDoS-атаки) и функциональные компоненты приложения (озвученные ранее веб-сервер, бекенд, база данных).

Также важным элементом быстродействия системы является кеширование несрочных запросов, позволяющее сократить количество обращений к некоторым элементам бекенда или к базе данных.

Было оговорено выше, что для минимизации задержек необходимо создание реплик контейнеров (веб-сервера и бекенда). Поэтому для равномерного распределения запросов между копиями приложению необходим такой компонент как балансировщик нагрузки. Он же может выступать и развязывающим звеном между пользователем и другими контейнерами, являясь единственной точкой входа в приложение.

Таким образом разрабатываемое приложение должно содержать следующие структурные элементы:

- Проксирующий элемент/балансировщик нагрузки;
- Веб-сервер (фронтенд);
- Логическая часть приложения (бекенд);
- Кеширующий элемент;
- База данных.

2.1.1 Взаимосвязь компонентов приложения

Для формирования архитектуры приложения необходимо рассмотреть конкретный пример отправки запроса приложению пользователем.

В первую очередь запрос должен быть принят первым балансировщиком нагрузки. На основании предыдущих запросов и своих алгоритмов он должен перенаправить данный запрос на один из веб-серверов, сам при этом никак не участвуя в формировании ответа.

Веб-сервер (фронтенд) должен по префиксу определить, пришёл ли запрос к статике или медиа файлам. Если да, веб-сервер должен

самостоятельно отдать соответствующие файлы, иначе же проксировать запрос далее.

Запрос к бекенду принимается вторым балансировщиком нагрузки, который распределяет его на один из сервисов бекенда. Бекенд выполняет всю работу по логической обработке запроса: формирует готовый ответ, отправляет кешированный до этого ответ, считывает информацию из базы данных, либо же заносит в неё новую.

Таким образом архитектуру можно представить в виде графа, изображенного на рисунке 1, где под «Other replicas» подразумевается произвольное количество дополнительных копий контейнеров.

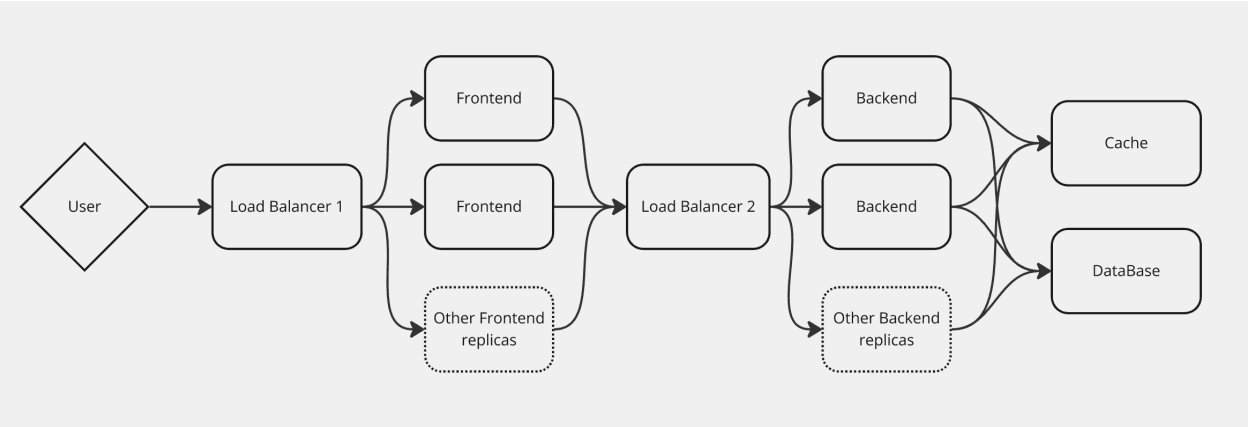

Рисунок 1 – Архитектура приложения

2.1.2 Выбор технологий реализации контейнеров

После определения списка структурных компонентов системы необходимо для каждого из них выбрать подходящие технологию/продукт реализации.

В качестве балансировщика нагрузки для контейнеризованных приложений можно рассмотреть как Traefik, так и Nginx. Оба инструмента хорошо зарекомендовали себя и активно используются в продакшн-средах. Nginx — это классический и мощный веб-сервер, который поддерживает балансировку нагрузки с использованием различных алгоритмов. Однако настройка Nginx требует вручную прописывать конфигурацию для каждого сервиса, что может быть неудобно при динамичном окружении в Docker.

Traefik, в свою очередь, предоставляет значительно более гибкую и автоматизированную настройку. Одним из ключевых преимуществ Traefik

является его способность автоматически обнаруживать контейнеры и их сервисы в Docker или Kubernetes без необходимости вручную изменять конфигурации. Traefik также обладает рядом встроенных функций, таких как автоматическая настройка SSL с использованием Let’s Encrypt, поддержка современных протоколов HTTP/2 и HTTP/3, а также возможность интеграции с различными системами мониторинга и логирования, что делает его более подходящим выбором для микросервисных архитектур и облачных приложений.

Кроме того, при сравнении скорости после усреднения результатов traefik показывает себя на 24% быстрее конкурента nginx [7].

Также nginx можно сравнить с apache, но уже в роли веб-сервера.

Apache — это зрелый и универсальный веб-сервер, поддерживающий широкий спектр модулей для расширения функциональности. Он особенно полезен для приложений, требующих гибкой настройки доступа и поддержки CGI. Однако Nginx превосходит Apache в производительности при работе с большим количеством одновременных соединений, что делает его более подходящим для современных высоконагруженных приложений. По результатам исследований nginx способен обслуживать запросы до 6 раз быстрее, при этом потребляя как минимум в 6 раз меньше ресурсов памяти [8].

Django — это мощный фреймворк для создания серверной части, идеально подходящий для разработки веб-приложений. Одним из ключевых преимуществ Django является встроенная система аутентификации и авторизации, которая обеспечивает готовые механизмы управления пользователями, создание групп, настройку ролей и обработку прав доступа. Эти возможности позволяют быстро внедрить надежную систему авторизации с минимальными усилиями, что критически важно для защиты данных и соблюдения современных стандартов безопасности [9].

Кроме того, Django предоставляет встроенную админ-панель, которая генерируется автоматически на основе моделей приложения. Эта панель позволяет разработчикам и администраторам управлять данными без необходимости создавать отдельный интерфейс. Помимо авторизации и администрирования, Django отличается высокой производительностью при

работе с базами данных благодаря мощной ORM, которая упрощает взаимодействие с данными. Немаловажную роль играет возможность быстрой интеграции REST модуля, позволяющего при необходимости сравнительно просто настроить REST взаимодействие в качестве API с другими (внешними) сервисами. А для ускорения работы у Django есть возможность использовать другие сервисы для кеширования (такие как Redis) [10].

Redis является отличным выбором для кеширования страниц благодаря высокой производительности, обеспечиваемой хранением данных в оперативной памяти и низким временем доступа. Система поддерживает механизмы автоматического удаления устаревших данных (TTL), что особенно важно для динамических приложений. Помимо кеширования страниц, Redis позволяет работать с различными структурами данных, такими как строки, хэш-таблицы и множества, что делает его универсальным инструментом. Благодаря лёгкой интеграции с популярными фреймворками, включая Django, и способности масштабироваться для обработки больших нагрузок, Redis идеально подходит для создания быстрых и надёжных систем кеширования [11].

Наиболее популярными решениями среди реляционных баз данных являются MySQL и PostgreSQL. У каждой из них есть свои особенности, достоинства и недостатки.

MySQL известна своей простотой и высокой производительностью в задачах, где требуется обработка множества простых запросов, что делает её подходящей для веб-приложений с интенсивным чтением данных. Однако её функциональность ограничена по сравнению с PostgreSQL, особенно в области сложных транзакций и работы с большими объёмами структурированных данных.

PostgreSQL выделяется своими мощными возможностями для обработки сложных запросов, поддержкой ACID и строгой консистентностью данных, что делает её подходящей для финансовых, научных и аналитических приложений [12].

Так как в разрабатываемом приложении необходимо с высокой надёжностью и безопасностью хранить и рассчитывать денежные суммы,

наиболее правильным выбором будет именно PostgreSQL. Также в пользу PostgreSQL говорит её более эффективная работа со сложными запросами, которые могут понадобиться для анализа выполняемых сотрудниками задач.

Таким образом был сформирован список конкретных используемых в каждом контейнере технологий и представлен на рисунке 2. Благодаря возможности создания отдельных маршрутов балансировки и проксирования стало возможно использовать лишь один контейнер traefik для распараллеливания контейнеров как фронтенда, так и бекенда.

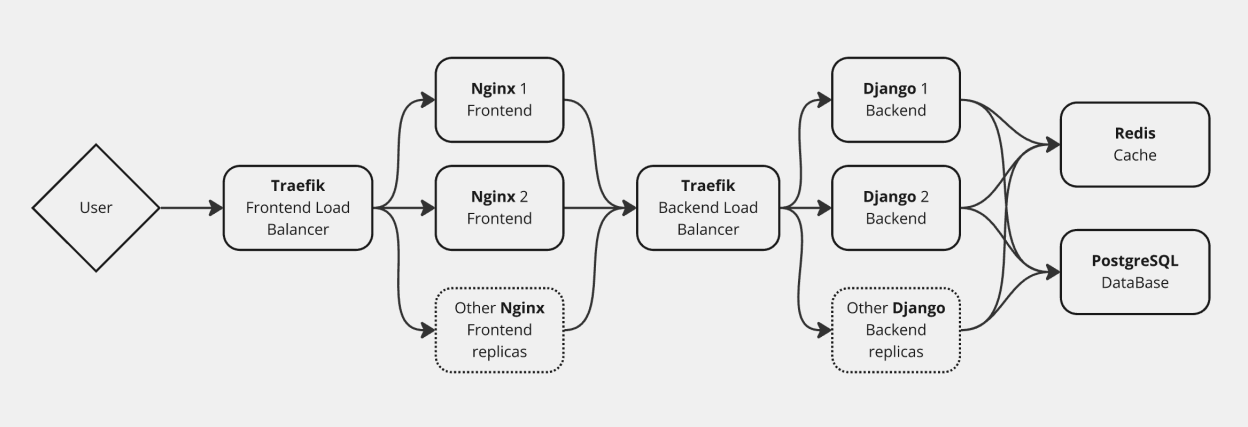

Рисунок 2 – Архитектура с технологиями реализации контейнеров

3 Реализация программного продукта

3.1 Практическая реализация продукта

3.1.1 Реализация архитектуры контейнеров

Для реализации разработанной в главе 2 архитектуры платформы автоматизации учёта рабочих часов и задач сотрудников необходимо в нужном порядке и с нужными конфигурациями (и конфигурационными файлами) запустить все контейнеры.

Для поднятия сразу множества различных контейнеров и их конфигурации необходимо в самом простом варианте воспользоваться инструментом Docker compose. Он позволяет в декларативном стиле описать желаемое состояние контейнеров в конфигурационном файле в формате yaml. После соответствующей команды Docker попытается поднять соответствующие контейнеры, применить к ним нужные настройки и собрать необходимые образы. В случае отсутствия ошибок все описанные контейнеры успешно поднимутся и свяжутся через стандартную Docker-сеть, позволяющую им взаимодействовать друг с другом через внутренние dns-имена, не позволяя взаимодействовать извне ни с какими контейнерами кроме тех, у которых явно будут прописаны проброшенные внешние порты.

Также возможно прописать количество желаемых реплик у контейнеров, которые должны быть размножены. Docker самостоятельно выполнит скаллирование, а поднятый traefik позволит связать реплики с единственной точкой входа в виде балансировщика нагрузки.

Также во избежание ошибок ненахождения одних контейнеров другими до их непосредственного поднятия необходимо установить правильный порядок поднятия контейнеров.

Самыми первыми в любом порядке должны быть подняты база данных и кеш хранилище, так как они ни к чему самостоятельно не подключаются и только принимают входящие соединения.

После этого необходимо поднять следующий независимый элемент — бекенд, который сможет подключиться к уже поднятым хранилищам, не требуя других соединений.

Следующим необходимо поднять контейнер веб-сервера, так как именно он соединяется с бекендом, проксируя на него запросы. Последним же должен быть поднят балансировщик нагрузки, который является единственной точкой входа в приложение. Даже если по какой-то причине во время поднятия контейнеров что-то пойдёт не так, из-за того что точка входа в приложение будет поднята в последнюю очередь, пользователям это показано никак не будет.

Таким образом была сформирована следующая очередность поднятия контейнеров:

1) Кеш (Redis);
2) База данных (PostgreSQL);
3) Бэкенд (Django);
4) Фронтенд (Nginx);
5) Балансировщик нагрузки (Traefik).

3.1.2 Реализация наполнения контейнеров

Наиболее простой конфигурацией, представленной на рисунке 3, из всех используемых контейнеров будет обладать именно кеш-хранилище Redis. Он не требует каких-либо специальных настроек, кроме указания образа и автоматического перезапуска при выключении. Полный исходный код файла конфигурации Docker-compose представлен в приложении Б.

```
83    redis_cache:
84      image: redis:latest
85      restart: always
```

Рисунок 3 – Конфигурация кеш-хранилища Redis

Следующей в порядке очереди будет база данных PostgreSQL, чья конфигурация представлена на рисунке 4. Для установки необходимых переменных окружения баз данных используется общий файл .env, содержимое которого представлено на рисунке 5. Также для возможности проверки корректного поднятия контейнера в его конфигурации запуска прописана проверка здоровья, запускающая внутреннюю команду диагностики с

определенным интервалом и позоляющая другим контейнерам (в данном случае бекенду) ориентироваться на результат выполнения этой команды.

Также важно заметить, что для сохранения данных хранилища после перезапуска необходимо в обязательном порядке создать средствами Docker-compose отдельную папку, которую база данных и будет использовать в роли хранилища данных.

```
29    postgres_db:
30      image: postgres:17
31
32      restart: always
33      env_file:
34        - .env
35
36      volumes:
37        - ./TaskAndTimes_db-data:/var/lib/postgresql/data
38
39      healthcheck:
40        test: [ "CMD-SHELL", "pg_isready -d ${POSTGRES_DB} -U ${POSTGRES_USER}" ]
41        interval: 10s
42        timeout: 5s
43        retries: 5
44        start_period: 10s
```

Рисунок 4 – Конфигурация базы данных PostgreSQL

```
1  POSTGRES_HOST=postgres_db
2  REDIS_CACHE_HOST=redis_cache
3
4  POSTGRES_DB=TaskAndTime_db
5  POSTGRES_USER=TaskAndTime_user
6  POSTGRES_PASSWORD=jH8gadBhb3B953FBafH908bv
7
8  DJANGO_SECRET_KEY='django-insecure-++^k%#$cehv(ztlo%0po-%vzx#yt$9akr_9&dez@7d1l$#)$_7'
9  HOST_IP=176.108.252.78
```

Рисунок 5 – Файл переменного окружения .env

Так как контейнеру бекенда Django нужно перед началом работы установить все необходимые зависимости, в файле конфигурации, представленной на рисунке 7, прописаны параметры сборки этого контейнера по информации из Dockerfile, содержимое которого представлено на рисунке 6.

```
1  FROM python:3.12
2
3  WORKDIR /usr/src/app
4
5  COPY requirements.txt ./
6  COPY .env ./
7
8  RUN python -m pip install --upgrade pip
9  RUN pip install --no-cache-dir -r requirements.txt
```

Рисунок 6 – Файл сборки образа бекенда Django

В конфигурации, представленной на рисунке 7, прописано создание 3-х реплик контейнера, монтирование директории с исходным кодом внутрь контейнера, установка рабочей директории, зависимости (с учетом проверки базы данных) и команда запуска прослушивания всех входящих запросов на порту 8000. Исходный код контейнера Django представлен в приложении А.

```yaml
4   services:
5     django_backend:
6       image: TaskMetrics/django:1
7
8       restart: always
9       deploy:
10        replicas: 3
11
12      build:
13        context: .
14        dockerfile: docker/django/Dockerfile
15
16      volumes:
17        - ./TaskAndTime:/usr/src/app/TaskAndTime
18
19      working_dir: /usr/src/app/TaskAndTime
20
21      depends_on:
22        postgres_db:
23          condition: service_healthy
24        redis_cache:
25          condition: service_started
26
27      command: [ 'gunicorn', '-b 0.0.0.0:8000', 'TaskAndTime.wsgi' ]
```

Рисунок 7 – Конфигурация бекенда Django

В конфигурации фронтенда nginx, представленной на рисунке 8, также также задано желаемое количество реплик контейнера, а также смонтированы папки статических и медиа файлов вместе с файлом настройки веб-сервера, содержимое которого представлено на рисунке 9.

Веб-сервер настроен на отдачу статических и медиа файлов при соответствующих запросах, а также на дальнейшее проксирование запроса на бекенд через балансировщик нагрузки в противном случае.

```
47    nginx_frontend:
48      image: nginx:1.27
49
50      restart: always
51      deploy:
52        replicas: 3
53
54      volumes:
55        - ./TaskAndTime/static:/srv/www/TaskAndTime/static
56        - ./TaskAndTime/media:/srv/www/TaskAndTime/media
57        - ./TaskAndTime/logs:/srv/www/TaskAndTime/logs
58        - ./docker/nginx:/etc/nginx/conf.d
59
60      depends_on:
61        - django_backend
```

Рисунок 8 – Конфигурация фронтенда Nginx

```
server {
  listen 8080;
  server_name localhost;
  charset     utf8;
  autoindex   off;
  set         $project_home /srv/www/TaskAndTime;


  location / {
    root $project_home;
    try_files $uri @Backend;
  }

  location @Backend {
    proxy_pass http://traefik_balancer:8090;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header is-backend 1;
  }
}
```

Рисунок 9 – Файл настройки контейнера Nginx

Конфигурация контейнера балансировщика нагрузки traefik, представленная на рисунке 10, содержит стартовые команды, указывающие контейнеру на расположение внутри него файлов статических и динамических настроек, представленных на рисунках 11 и 12 соответственно. Также эти файлы смонтированы внутрь контейнера.

В файле статической настройки балансирощика указаны точки входа: для внешних пользователей и для внутренних контейнеров; параметры запуска с панелью управления в отладочном режиме, а также указаны необходимые параметры для связи с реестром Docker и файла динамических параметров.

В файле динамических параметров прописаны маршрутизаторы для входящих запросов от пользователей и от контейнеров фронтенда. Первые обслуживаются сервисом, распределяющим нагрузку между фронтендами, вторые — сервисом, распределяющим запросы между бекендами.

```yaml
63    traefik_balancer:
64      image: traefik:v3.2
65
66      ports:
67        - 80:80
68        - 8080:8080
69
70      command:
71        - "--configFile=/etc/traefik/traefik.yml"
72        - "--providers.file.filename=/etc/traefik/dynamic.yml"
73
74      volumes:
75        - /var/run/docker.sock:/var/run/docker.sock
76        - ./docker/traefik/traefik.yml:/etc/traefik/traefik.yml
77        - ./docker/traefik/dynamic.yml:/etc/traefik/dynamic.yml
78
79      depends_on:
80        - django_backend
81        - nginx_frontend
```

Рисунок 10 – Конфигурация балансировщика нагрузки Traefik

```yaml
entryPoints:
  web:
    address: ":80"
  backend:
    address: ":8090"

api:
  dashboard: true
  insecure: true

providers:
  docker:
    endpoint: "unix:///var/run/docker.sock"
    exposedByDefault: false

  file:
    filename: "/etc/traefik/dynamic.yml"
    watch: true
```

Рисунок 11 – Файл статической конфигурации traefik

```yaml
http:
  routers:
    nginx:
      rule: "PathPrefix(`/`)"
      service: frontend_service
      entryPoints:
        - web

    django:
      rule: "Header(`is-backend`, `1`)"
      service: backend_service
      entryPoints:
        - backend

  services:
    frontend_service:
      loadBalancer:
        servers:
          - url: "http://nginx_frontend:8080"

    backend_service:
      loadBalancer:
        servers:
          - url: "http://django_backend:8000"
```

Рисунок 12 – Файл динамической конфигурации traefik

3.2 Теоретическая реализация продукта

Существует теоретическая возможность реализации разработанного приложения на кластере Kubernetes вместо Docker-compose для увеличения стабильности работы под высокой нагрузкой, а также настройки CI/CD для возможности продолжения непрерывной разработки продукта. Практически

данные возможности реализованы не были, поэтому далее описаны будут теоретически.

Развертывание приложения в кластере Kubernetes может быть целесообразным, если требуется обеспечить масштабируемость и отказоустойчивость системы. Kubernetes позволяет автоматически управлять нагрузкой между контейнерами и горизонтально масштабировать приложение, добавляя или убирая экземпляры контейнеров в зависимости от нагрузки. Кластер также предоставляет механизмы автоматического перезапуска контейнеров в случае их отказа, что повышает надежность системы и минимизирует простои [13].

Кроме того, использование Kubernetes обеспечивает централизованное управление инфраструктурой и автоматизацию жизненного цикла приложения. С его помощью можно настроить сложные сценарии деплоя, что позволяет минимизировать риски при выпуске обновлений. Kubernetes также обеспечивает изоляцию между сервисами, эффективное распределение ресурсов и интеграцию с экосистемой облачных сервисов, таких как балансировщики нагрузки, системы мониторинга и сетевые политики, что делает его оптимальным решением для сложных и масштабируемых приложений.

Настройка CI/CD для разрабатываемого приложения может значительно упростить и ускорить процесс разработки и развертывания. Интеграция CI/CD позволяет автоматизировать этапы сборки, тестирования и доставки приложений, что минимизирует человеческий фактор и снижает вероятность ошибок. Например, каждое внесение изменений в код автоматически запускает проверку на ошибки, тестирование и сборку нового контейнера. Это обеспечивает быстрый и безопасный выпуск обновлений, так как проблемы выявляются на раннем этапе разработки [14].

Кроме того, CI/CD способствует ускорению процессов доставки продукта в рабочую среду. После успешного прохождения всех тестов приложение может быть автоматически развернуто в различных окружениях. Это гарантирует, что разработчики смогут быстро вносить изменения и обновления, не тратя время на ручные операции. CI/CD также обеспечивает

прозрачность и стандартизацию всех процессов, что делает его особенно ценным для масштабируемых и долгосрочных проектов.

3.3 Инструкция к развертыванию приложения

Для успешного деплоя приложения необходимо получить доступ к выделенному персональному серверу с установленными на него unix-подобной системой и ядром Docker Engine.

После подключения к VPS по ssh необходимо первым делом скопировать исходный код проекта на его хранилище, либо же склонировать репозиторий из любой подходящей системы контроля версий командой:

```
git clone <repository-url>
```

После этого необходимо в корень скопированного проекта любым удобным способом перенести файл переменного окружения .env (если он не был скопирован из-за его отсутствия в системе контроля версий) командой:

```
scp .env <vps-host>@<vps-ip>:~/<project-name>.
```

Далее для окончательного развертывания необходимо выполнить команду:

```
docker compose up -d --build.
```

После этого на порту 80 выделенного сервера станет доступно приложение. Далее необходимо применить миграции базы данных Django командой:

```
docker compose exec django_backend python manage.py migrate
```

после чего, при необходимости, создать суперпользователя командой:

```
docker compose exec django_backend python manage.py
createsuperuser
```

и затем проследовать инструкции программы.

Пример деплоя платформы автоматизации учёта рабочих часов и задач сотрудников представлен на рисунках 13 и 14.

```
arsenoks4132@my-test-vps: ~/ | Windows PowerShell

Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/pscore6)

PS C:\Users\Danov> cd .\PycharmProjects\TaskMetrics\
PS C:\Users\Danov\PycharmProjects\TaskMetrics> ls


    Каталог: C:\Users\Danov\PycharmProjects\TaskMetrics


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        09.12.2024      3:40                .idea
d-----        30.11.2024     14:36                .venv
d-----        07.12.2024     10:46                docker
d-----        08.12.2024     13:32                TaskAndTime
da----        09.12.2024      2:57                TaskAndTimes_db-data
-a----        09.12.2024      3:24            279 .env
-a----        08.12.2024     13:32             69 .gitignore
-a----        09.12.2024      3:23           1807 docker-compose.yaml
-a----        09.12.2024      2:56            372 requirements.txt


PS C:\Users\Danov\PycharmProjects\TaskMetrics> scp .env arsenoks4132@176.108.252.78:~/TaskMetrics
.env                                                                                  100%  279    45.4KB/s   00:00

PS C:\Users\Danov\PycharmProjects\TaskMetrics>
```

Рисунок 13 – Копирование файла переменного окружения .env

```
arsenoks4132@my-test-vps: ~ | Windows PowerShell

arsenoks4132@my-test-vps:~$ git clone -b docker https://github.com/Arsenoks4132/TaskMetrics.git
Cloning into 'TaskMetrics'...
remote: Enumerating objects: 415, done.
remote: Counting objects: 100% (415/415), done.
remote: Compressing objects: 100% (262/262), done.
remote: Total 415 (delta 141), reused 389 (delta 119), pack-reused 0 (from 0)
Receiving objects: 100% (415/415), 480.82 KiB | 2.70 MiB/s, done.
Resolving deltas: 100% (141/141), done.
arsenoks4132@my-test-vps:~$ cd TaskMetrics/
arsenoks4132@my-test-vps:~/TaskMetrics$ docker compose up -d --build
[+] Running 9/9
 ✓redis_cache Pulled
   ✓bc0965b23a04 Already exists
   ✓9501a6ec095f Pull complete
   ✓98e7597530ef Pull complete
   ✓75dffa679c9b Pull complete
   ✓8912a88e73c8 Pull complete
   ✓141f00d6fee8 Pull complete
   ✓4f4fb700ef54 Pull complete
   ✓8242f9d5b464 Pull complete
[+] Building 41.2s (12/12) FINISHED
 => [django_backend internal] load build definition from Dockerfile
 => => transferring dockerfile: 209B
 => [django_backend internal] load metadata for docker.io/library/python:3.12
 => [django_backend internal] load .dockerignore
 => => transferring context: 2B
 => [django_backend 1/6] FROM docker.io/library/python:3.12@sha256:2fc3687451585d73c06624d54690baf71c61cc2a144549b5b7dbca60048a5fb2
 => [django_backend internal] load build context
 => => transferring context: 731B
 => CACHED [django_backend 2/6] WORKDIR /usr/src/app
 => [django_backend 3/6] COPY requirements.txt ./
 => [django_backend 4/6] COPY .env ./
 => [django_backend 5/6] RUN python -m pip install --upgrade pip
 => [django_backend 6/6] RUN pip install --no-cache-dir -r requirements.txt
 => [django_backend] exporting to image
 => => exporting layers
 => => writing image sha256:0456cccdab40fbfb933d9e3c3da25c7fb5fa8b817245a06038f752f401d9a7e0
 => => naming to TaskMetrics/django:1
 => [django_backend] resolving provenance for metadata file
[+] Running 10/10
 ✓Network taskmetrics_default                Created
 ✓Container taskmetrics-postgres_db-1        Healthy
 ✓Container taskmetrics-redis_cache-1        Started
 ✓Container taskmetrics-django_backend-3     Started
 ✓Container taskmetrics-django_backend-1     Started
 ✓Container taskmetrics-django_backend-2     Started
 ✓Container taskmetrics-nginx_frontend-3     Started
 ✓Container taskmetrics-nginx_frontend-1     Started
 ✓Container taskmetrics-nginx_frontend-2     Started
 ✓Container taskmetrics-traefik_balancer-1   Started
arsenoks4132@my-test-vps:~/TaskMetrics$
```

Рисунок 14 – Развертывание приложения

3.4 Демонстрация работоспособности приложения

В ходе разработки платформы был реализован минимальный необходимый функционал, позволяющий на базовом уровне использовать полученный продукт по прямому назначению. Сотрудникам после регистрации становится доступна страница личного профиля, представленная на рисунке 15, позволяющая добавить выполненную задачу в базу данных приложения.

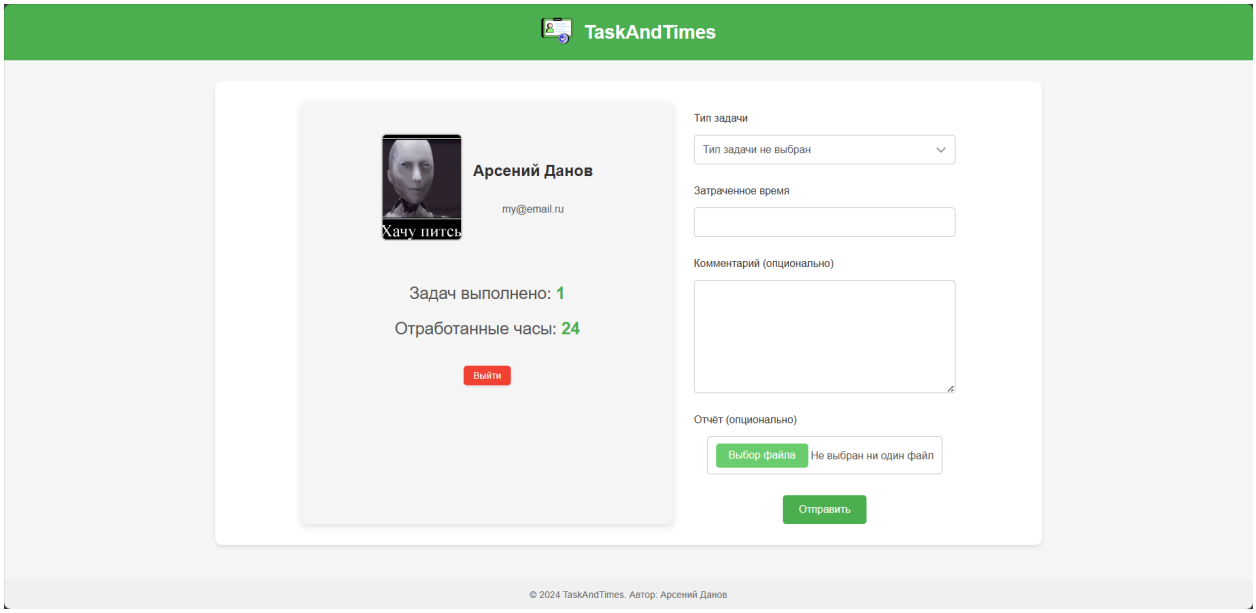

Рисунок 15 – Страница профиля сотрудника

Руководителям помимо стандартной страницы профиля доступна страница общей статистики, представленная на рисунке 16, на которой собрана информация по всем сотрудникам сразу. При нажатии же на конкретного сотрудника руководителю становится доступна страница с подробной информацией по конкретному сотруднику, представленная на рисунке 17.

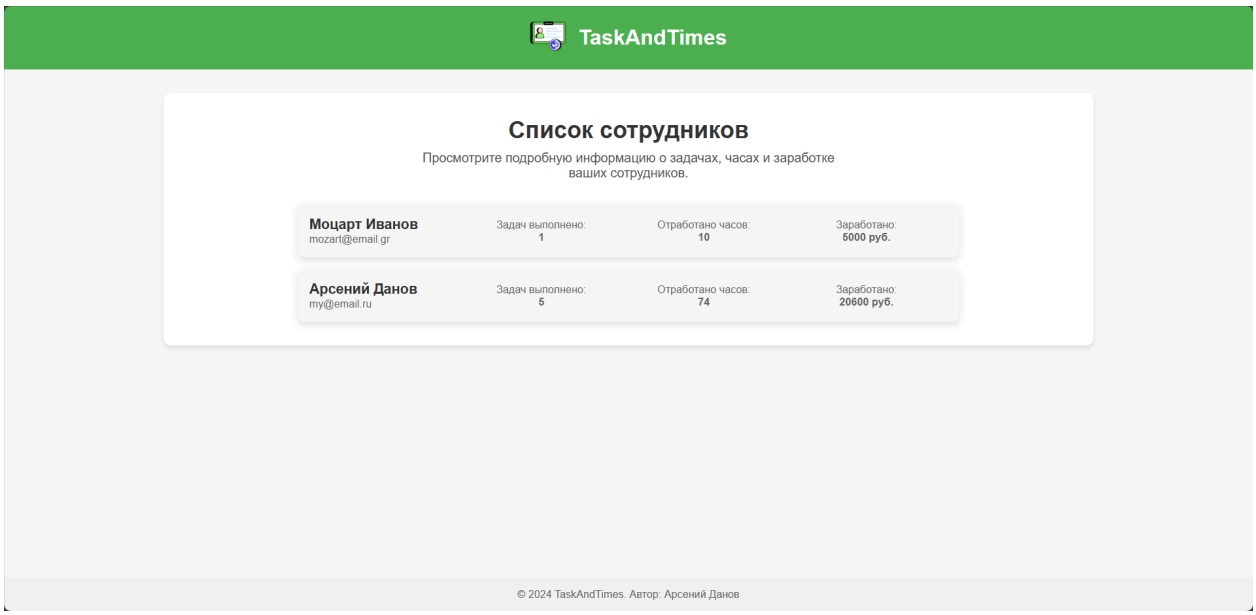

Рисунок 16 – Страница с информацией по всем сотрудникам

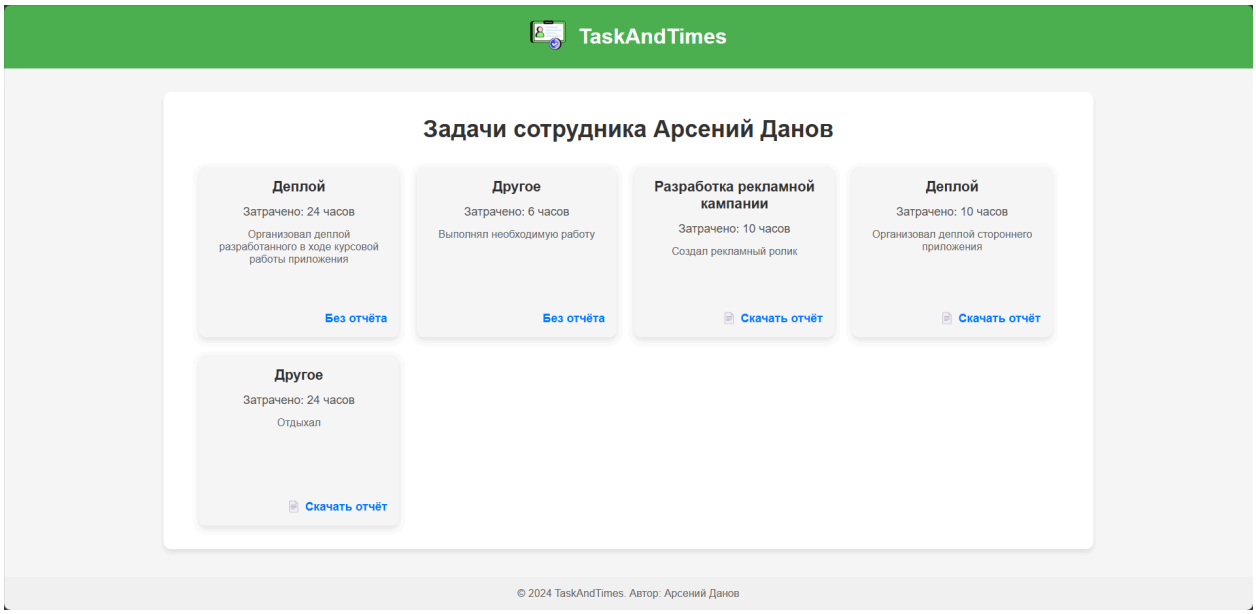

Рисунок 17 – Страница с подробной информацией по конкретному сотруднику

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсовой работы по настройке и администрированию платформы для автоматизации учёта рабочих часов и задач сотрудников была разработана система, которая использует несколько современных технологий для обеспечения высокой производительности и надёжности. В качестве бэкенда был выбран Django, что обеспечило удобную и масштабируемую обработку данных, управление пользователями и задачами. Для проксирования запросов и отдачи статических файлов использовался Nginx, который эффективно справляется с высокими нагрузками и обеспечивает быструю отдачу контента.

Для хранения данных была выбрана PostgreSQL, что гарантировало надёжность и масштабируемость базы данных, а также возможность работы с большими объёмами информации. Для кеширования часто запрашиваемых данных был использован Redis, что повысило скорость отклика приложения. Кроме того, для управления трафиком между компонентами и обеспечения отказоустойчивости системы был внедрён Traefik. Приложение было развернуто на виртуальном приватном сервере (VPS), что позволило гибко настроить инфраструктуру и эффективно контролировать её работу.

Исходный код разработанного приложения размещен доступен по ссылке: https://github.com/Arsenoks4132/TaskMetrics

Развернутое на VPS приложение доступно по ссылке: http://176.108.252.78

Для удобства проверки в базе данных развернутого приложения был создан пользователь с именем пользователя test_user и паролем 1234, обладающий минимальными необходимыми правами доступа для проверки работоспособности функционала для руководителей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Болбаков Р.Г., Матчин В.Т., Мордвинов В.А. Настройка и администрирование сервисного программного обеспечения [Электронный ресурс]: учебное пособие ISBN 978–5–7339–1860–0. Москва: РТУ МИРЭА, 2023. 142 с. URL: https://e.lanbook.com/book/382442 (дата обращения: 09.12.2024).
2. Stack Overflow Developer Survey 2024 [Электронный ресурс]: опрос. 2024. URL: https://survey.stackoverflow.co/2024/technology (дата обращения: 09.12.2024).
3. Маркелов А.А. Введение в технологию контейнеров и Kubernetes [Электронный ресурс]: ISBN 978–5–97060–775–6. Москва: ДМК Пресс, 2019. 194 с. URL: https://e.lanbook.com/book/131702 (дата обращения: 09.12.2024).
4. Дешко И.П., Кряженков К.Г. Управление сетевыми информационными системами [Электронный ресурс]: курс лекций. Москва: РТУ МИРЭА, 2021. 174 с. URL: https://e.lanbook.com/book/176536 (дата обращения: 09.12.2024).
5. Миронов А.Н., Воронцов Ю.А., Михайлова Е.К. Системное программное обеспечение [Электронный ресурс]: учебное пособие. Москва: РТУ МИРЭА, 2022. 216 с. URL: https://e.lanbook.com/book/265712 (дата обращения: 09.12.2024).
6. Годзурас Э. Docker Compose для разработчика [Электронный ресурс]: руководство ISBN 978–5–93700–203–7. Москва: ДМК Пресс, 2023. 220 с. URL: https://e.lanbook.com/book/348110 (дата обращения: 09.12.2024).
7. Johansson A. HTTP Load Balancing Performance Evaluation of HAProxy, NGINX, Traefik and Envoy with the Round-Robin Algorithm. 2022.
8. Chihana S. Web Server Performance of Apache and Nginx: A Systematic Literature Review. 2018.
9. Персиваль Г. Python. Разработка на основе тестирования. Повинуйся Билли-тестировщику, используя Django, Selenium и JavaScript [Электронный ресурс]: ISBN 978–5–97060–594–3. Москва: ДМК Пресс,

2018. 622 с. URL: https://e.lanbook.com/book/111440 (дата обращения: 09.12.2024).

10. Меле А. Django 4 в примерах [Электронный ресурс]: руководство ISBN 978–5–93700–204–4. Москва: ДМК Пресс, 2023. 800 с. URL: https://e.lanbook.com/book/348113 (дата обращения: 09.12.2024).
11. Годзурас Э. Docker Compose для разработчика [Электронный ресурс]: руководство ISBN 978–5–93700–203–7. Москва: ДМК Пресс, 2023. 220 с. URL: https://e.lanbook.com/book/348110 (дата обращения: 09.12.2024).
12. PipeOps. PostgreSQL and MySQL: Which is Better? A 2024 Comparison. 2024.
13. Вьяс Д., Лав К. Kubernetes изнутри [Электронный ресурс]: ISBN 978–5–93700–153–5. Москва: ДМК Пресс, 2023. 378 с. URL: https://e.lanbook.com/book/314942 (дата обращения: 09.12.2024).
14. Баланов А.Н. Комплексное руководство по разработке: от мобильных приложений до веб-технологий [Электронный ресурс]: учебное пособие для СПО ISBN 978–5–507–48841–4. 3-е изд. Санкт-Петербург: Лань, 2024. 412 с. URL: https://e.lanbook.com/book/438890 (дата обращения: 09.12.2024).